StudySync — Peer Matching & Collaborative Study Group Platform

Software Requirements Specification (SRS)

Standard: IEEE Std 830-1998

Version: 1.0.0 (Mid-Semester Formal Release)

Status: Approved & Implemented Baseline

Course: Software Engineering & Project Management

Project Group: Group 15 (Section F)

Target System: StudySync Collaborative Web Platform

Date of Release: October 16, 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete software, functional, and non-functional requirements for **StudySync (Version 1.0.0)**. It serves as the authoritative technical baseline for students, developers, quality assurance testers, academic project supervisors, and institutional evaluators during the mid-semester evaluation milestone of the Software Engineering Project.

1.2 Document Conventions

- **Requirement Identifiers:**
  - Functional requirements are denoted as `FR-XXX` (e.g., `FR-001`).
  - Non-functional requirements are denoted as `NFR-XXX` (e.g., `NFR-001`).
  - Use cases are denoted as `UC-XX` (e.g., `UC-01`).
  - Test cases are denoted as `TC-XXX` (e.g., `TC-001`).
- **Typographical Standards:** Code identifiers, database tables, and API routes are formatted in `monospace`.
- **Prioritization Tagging:** Requirements are prioritized following the MoSCoW convention (**Must Have**, **Should Have**, **Could Have**, **Won't Have**).

1.3 Intended Audience & Reading Suggestions

1. **Academic Evaluators & Supervisors:** Focus on Section 1.4 (Scope), Section 4 (Functional Requirements), and Section 6 (Traceability Matrix).
2. **Developers (Frontend & Backend):** Refer to Section 3 (External Interfaces), Section 4 (System Features), and Section 5 (Non-Functional Requirements).
3. **Quality Assurance & Testers:** Utilize Section 4 acceptance criteria and Section 5.1/5.3 for verifying test registers.

1.4 Product Scope

StudySync is a specialized, course-centric web application designed to eliminate the friction higher education students encounter when forming and sustaining academic study groups.
The software enables students to:

- Authenticate securely and build an academic profile with majors, learning goals, and recurring weekly time availability.
- Discover and filter study groups aligned with specific university courses and subject codes.
- Compute peer and group compatibility via a deterministic, explainable multi-variable algorithm.
- Manage membership requests with owner approval workflows and group capacity boundaries.
- Coordinate study sessions with automated calendar collision checking and attendance RSVPs.
- Centralize course notes, repository links, and study materials in access-controlled groups.

1.5 References

1. IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
2. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
3. PostgreSQL Global Development Group. (2024). *PostgreSQL 14.x Documentation*.
4. Express.js Project. (2024). *Express 4.x API Specification*.
5. React Core Documentation. (2024). *React 18 Concurrent Rendering & State Management*.

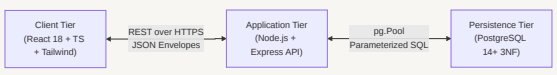
---

2. Overall Description

2.1 Product Perspective

StudySync is an autonomous, self-contained Three-Tier Web Application:

- **Presentation Tier (Client):** Single Page Application (SPA) built using React 18, TypeScript, and Tailwind CSS, bundled with Vite.
- **Application Tier (Server):** Modular RESTful API service built on Node.js and Express.js, featuring Zod schema validation and stateless JWT authentication.
- **Persistence Tier (Database):** PostgreSQL 14+ relational database operating in Third Normal Form (3NF) with JSONB temporal extensions.



2.2 Product Functions Summary

- **Identity & Profiles:** Student onboarding, email verification, password hashing, academic goals, and weekly recurring schedule management.
- **Group Catalog:** Subject-indexed study groups with real-time membership counts and tags.
- **Matching Engine:** Multi-factor compatibility scoring based on Subject (40%), Schedule Overlap (40%), and Goals (20%).
- **Governance:** Applicant submission notes, owner approval/rejection panel, and capacity guards ($2 \leq n \leq 50$).
- **Scheduling & Calendar:** Session scheduling, attendee collision detection, and calendar RSVP states.
- **Resources:** Group-scoped links, syllabi, notes, and study material repository.

2.3 User Classes and Characteristics

1. **Unauthenticated Visitor:**
   - *Technical Expertise:* Low to Moderate.
   - *Privileges:* View public landing page, read feature summaries, register an account, or log in.
2. **Authenticated Student (Standard User):**
   - *Technical Expertise:* Moderate.
   - *Privileges:* Maintain student profile, specify weekly availability, search public study groups, submit join applications, view recommended groups, RSVP to sessions, access shared resources.
3. **Study Group Owner / Admin:**
   - *Technical Expertise:* Moderate.
   - *Privileges:* All student privileges, plus: create study groups, edit group metadata, review pending applicant profiles, approve/reject members, schedule group sessions, and upload resources.
4. **System Administrator:**
   - *Technical Expertise:* High.
   - *Privileges:* Platform maintenance, database integrity oversight, and catalog subject curation.

2.4 Operating Environment

- **Client Platforms:** Modern desktop and mobile web browsers:
  - Google Chrome (Version 110+)
  - Mozilla Firefox (Version 110+)
  - Apple Safari (Version 16+)
  - Microsoft Edge (Version 110+)
- **Server Platform:** Node.js LTS (v18.x or v20.x or v24.x) running on Linux (Ubuntu 22.04+ LTS), Windows Server / Windows 11, or macOS.
- **Database Engine:** PostgreSQL 14.0 or higher.
- **Hardware Minimums:** Server: 1 vCPU, 1 GB RAM, 10 GB SSD. Client: Any device capable of rendering modern HTML5/Canvas DOM.

2.5 Design and Implementation Constraints

- **Security Mandate:** Cleartext passwords shall never be persisted or logged; passwords must be salted and hashed via bcrypt (cost factor 10+).
- **Zero Secret Exposure:** Database passwords, JWT secret keys, and environment variables must never be checked into version control.
- **Relational Integrity:** All data entities must reside in Third Normal Form (3NF) with strict primary and foreign key constraints.
- **Standardized Communication:** All API interactions must utilize HTTP REST conventions with uniform JSON response envelopes (`{ success, data, error, meta }`).

2.6 User Documentation

- Comprehensive README located at project root ([`README.md`](file:///c:/Users/NAGARJUNA/Desktop/StudySync/README.md)) providing step-by-step local execution instructions.
- Interactive API documentation specification located at [`docs/api/API_SPECIFICATION.md`](file:///c:/Users/NAGARJUNA/Desktop/StudySync/docs/api/API_SPECIFICATION.md).

2.7 Assumptions and Dependencies

- Users have reliable internet access to interact with the web client.
- Students possess valid institutional email credentials (`@university.edu`).
- The underlying PostgreSQL server is accessible via local TCP or secure socket connection.

---

3. External Interface Requirements

3.1 User Interfaces

- **Responsive Layout:** Fluid layout built with Tailwind CSS supporting viewport resolutions from 360px (mobile) to 1920px (widescreen desktop).
- **Navigation Structure:** Persistent top navigation bar containing branding, direct links to "Explore Groups", "Recommendations", "My Schedule", and a profile dropdown.
- **Visual Feedback:** All asynchronous operations (form submissions, searches) render animated loading spinners, toast alerts for errors/success, and disabling states to prevent double submission.

3.2 Hardware Interfaces

- No direct custom hardware interfaces required. The system operates entirely over standard IP networks using TCP/IP.

3.3 Software Interfaces

- **Node.js Runtime & Express.js:** Serves REST API endpoints on port 5000 (configurable via `PORT` environment variable).
- **PostgreSQL Database:** Interfaced through the `pg` connection pool library on standard port 5432 with parameterized queries.
- **Operating System:** Platform-agnostic (POSIX-compliant Linux/macOS and Windows environments).

3.4 Communications Interfaces

- **Protocol:** HTTP/1.1 and HTTPS.
- **Data Exchange Format:** Standard JSON (`Content-Type: application/json`).
- **Authorization Header:** `Authorization: Bearer <JWT_TOKEN>`.
- **API Envelope Contract:**

```
{
  "success": true,
  "data": { ... },
  "error": null,
  "meta": { "timestamp": "2026-10-16T12:00:00.000Z" }
}
```

---

4. System Features & Functional Requirements

4.1 Module 1: Authentication & User Accounts

Description

Provides secure student registration, credential validation, stateless session token management, and account identity retrieval.

Functional Requirements

- **FR-001 (Registration):**
  - The system shall allow an unauthenticated student to create an account by providing full name, institutional email, and password.
  - The system shall enforce email uniqueness against existing active accounts.
  - The system shall validate that password length is at least 8 characters and contains at least 1 uppercase letter, 1 digit, and 1 special symbol.
- **FR-002 (Authentication & JWT Issuance):**
  - The system shall authenticate registered users by verifying supplied email and password against stored cryptographic hashes.
  - Upon successful verification, the system shall issue a signed JWT token containing `userId`, `email`, and `role`, signed with HMAC-SHA256 and configured with a 24-hour expiration window.
- **FR-003 (Password Hashing):**
  - The system shall cryptographically hash passwords using `bcrypt` with a minimum salt work factor of 10 prior to persistence in the `users` table.
  - Passwords shall never be returned in API query responses or logged in server telemetry.

4.2 Module 2: Student Academic Profiles & Weekly Availability

Description

Allows students to declare academic characteristics, focus areas, and recurring free time slots to enable accurate peer matching.

Functional Requirements

- **FR-004 (Profile Metadata Configuration):**
  - The system shall allow authenticated students to view and update their profile details, including university major, academic year/cohort, biography text, and learning goals array.
- **FR-005 (Weekly Availability Specification):**
  - The system shall permit students to specify recurring weekly free time slots.
  - Each time slot shall contain `dayOfWeek` (0–6, Sunday to Saturday), `startTime` (HH:MM in 24h format), and `endTime` (HH:MM in 24h format).
  - The system shall enforce that `endTime` is strictly greater than `startTime`.

- Availability shall be stored in PostgreSQL as structured `JSONB` to enable fast schedule intersection queries.

4.3 Module 3: Study Group Creation & Course Catalog Directory

Description

Enables students to establish academic study groups linked to university subjects and provides an indexed directory for searching and browsing active groups.

Functional Requirements

- **FR-006 (Study Group Creation):**
  - The system shall permit an authenticated student to create a study group by specifying group name, description, subject UUID, maximum member capacity ($2 \leq n \leq 50$), and visibility (`public` or `private`).
  - Upon group creation, the system shall automatically create a membership record enrolling the creator with role `'owner'` and status `'accepted'`.
- **FR-007 (Catalog Search and Filtering):**
  - The system shall provide an endpoint allowing students to query study groups with optional filters for subject code, search keywords, and availability of open slots.
  - The catalog response shall include real-time counts of accepted active members and owner profile details.
  - The query shall paginate results with configurable `page` and `limit` parameters (defaulting to 10 items per page).

4.4 Module 4: Membership Request, Review & Approval Lifecycle

Description

Implements a controlled application and admission workflow allowing study group owners to curate group composition and enforce size limits.

Functional Requirements

- **FR-008 (Join Request Submission):**
  - The system shall allow an authenticated student to submit an application to join a public study group with an optional introductory motivation note.
  - The system shall reject duplicate applications from students who already hold a `pending` or `accepted` membership with HTTP 409 Conflict.
- **FR-009 (Owner Review & Decision):**
  - The system shall provide an endpoint for group owners to retrieve all pending membership applications for their group.
  - The system shall permit the group owner to update an applicant's status to `'accepted'` or `'rejected'`.
- **FR-010 (Capacity Guard & Concurrency Control):**
  - When an owner attempts to approve an applicant, the system shall check that the current number of accepted members is strictly less than `max_members`.
  - If the group has reached full capacity, the system shall block the approval, return HTTP 400 Bad Request (`GROUP_AT_CAPACITY`), and maintain data integrity.

4.5 Module 5: Peer & Group Algorithmic Compatibility Engine

Description

Computes a transparent, explainable compatibility rating between a student and prospective study groups to optimize group formation.

Functional Requirements

- **FR-011 (Algorithmic Compatibility Formula):**
  - The system shall calculate an explainable compatibility score (0–100%) between a student and a study group using a three-tier weighted formula:

$$\text{Score} = (S_{\text{subject}} \times 0.40) + (S_{\text{schedule}} \times 0.40) + (S_{\text{goals}} \times 0.20)$$

  - *Subject Alignment Score (S_{subject}):* 100% if the group's subject matches any of the student's preferred subjects; 0% otherwise.
  - *Schedule Overlap Score (S_{schedule}):* Computed by calculating the total overlapping minutes between the student's recurring weekly availability and the group's collective active member availability slots, normalized up to a 4-hour weekly baseline.
  - *Goals Semantic Overlap (S_{goals}):* Computed via Jaccard token overlap between student learning goal keywords and group tags/description.
- **FR-012 (Ranked Recommendation Feed):**
  - The system shall return a list of public groups sorted in descending order of compatibility score.
  - The response shall return an explainable breakdown object `{ subjectScore, scheduleScore, goalsScore, totalScore }`.

4.6 Module 6: Study Session Scheduling & Overlap Collision Detection

Description

Allows group leaders to organize structured study meetings with automated schedule conflict warnings.

Functional Requirements

- **FR-013 (Study Session Creation):**
  - The system shall permit group owners and admins to create study sessions specifying title, description, start timestamp, end timestamp, location type (`online` or `in_person`), and meeting link or classroom venue.
- **FR-014 (Temporal Interval Validation):**
  - The system shall validate that session `end_time` is chronologically after `start_time`.
  - The system shall reject scheduling requests where `start_time` is in the past.
- **FR-015 (Collision Detection Guard):**
  - Prior to session creation, the system shall query all scheduled sessions of enrolled group members across all groups they belong to.
  - If a time overlap is detected for one or more members, the system shall return an advisory warning detailing the conflicting members and session times.

4.7 Module 7: Interactive Calendar View & RSVP Tracking

Description

Provides a calendar dashboard for students and tracks member attendance commitments.

Functional Requirements

- **FR-016 (Calendar Aggregation View):**
  - The system shall provide an endpoint returning all scheduled sessions for groups the student is an accepted member of, filtered by month, week, or day.
- **FR-017 (Session RSVP State):**
  - The system shall permit enrolled group members to submit their attendance RSVP status (`attending`, `declined`, `tentative`).
- **FR-018 (Attendance Check-In Verification):**
  - The system shall allow meeting organizers or attendees to record a timestamped check-in during the active window of the study session.

4.8 Module 8: Centralized Academic Resource Repository

Description

Enables authenticated group members to share and organize academic materials securely.

Functional Requirements

- **FR-019 (Resource Sharing):**
  - The system shall permit group members to share study resources (URLs, GitHub repositories, lecture slide links, Markdown summaries) associated with a study group.
- **FR-020 (Resource Access Control):**
  - The system shall restrict read and download access for a group's resources exclusively to authenticated members of that specific study group (`HTTP 403 Forbidden` for non-members).

---

5. Other Non-Functional Requirements

5.1 Performance Requirements

- **NFR-001 (API Response Latency):** 95% of read requests (`GET /api/v1/groups`, `GET /api/v1/recommendations/groups`) shall execute in $< 200\,\text{ms}$ under a load of 50 concurrent requests.
- **NFR-002 (Connection Pool Management):** The backend database driver shall maintain a pool of 2 to 10 connections, releasing idle connections within 30 seconds to prevent database resource exhaustion.
- **NFR-003 (Client Bundle Optimization):** The production frontend JavaScript bundle generated by Vite shall not exceed 350 KB compressed (gzip).

5.2 Safety and Disaster Resilience

- **NFR-004 (Transactional Atomicity):** Multi-step mutations (e.g., membership approval + capacity checks) shall execute within atomic database transactions (`BEGIN ... COMMIT`) to ensure that failures never leave orphaned or inconsistent states.
- **NFR-005 (Graceful Server Recovery):** Uncaught runtime exceptions shall be intercepted by top-level middleware, returning clean HTTP 500 error envelopes to the client without crashing the Node.js process.

5.3 Security and Privacy Requirements

- **NFR-006 (Credential Protection):** User passwords shall be salted and hashed with `bcrypt` (10 rounds). Cleartext passwords shall never be saved, transmitted in logs, or returned in API responses.
- **NFR-007 (SQL Injection Immunity):** All database operations in repositories shall exclusively use parameterized queries with placeholder tokens (`$1, $2, ...`). Dynamic SQL string concatenation is strictly prohibited.
- **NFR-008 (Payload Sanitization):** All incoming HTTP request payloads must match declarative Zod schemas, stripping unexpected fields and rejecting invalid formats prior to domain logic execution.
- **NFR-009 (Role-Based Access Control):** Protected endpoints shall enforce strict permission checks verifying that the requesting user holds the required role (`owner`, `member`, `admin`).
- **NFR-010 (Zero Credentials in Git):** Zero API keys, JWT secrets, or database passwords shall be committed to git repositories; all configuration shall be injected via `.env` files governed by `.env.example`.

5.4 Software Quality Attributes

- **Usability:** User interface elements shall maintain a minimum color contrast ratio of 4.5:1 (WCAG 2.1 AA compliant) and provide immediate user feedback on all operations.
- **Portability:** The client web application shall render uniformly across mobile viewports (360px) and desktop monitors (1920px) without horizontal scrolling.
- **Maintainability:** Codebase structure shall enforce clean separation of concerns: Routes $\rightarrow$ Controllers $\rightarrow$ Services $\rightarrow$ Repositories.

- **Testability:** 100% of functional requirements must have corresponding automated or manual test cases in the test register.

5.5 Business and Concurrency Integrity Rules

- **Rule 1 (Capacity Bounds):** A study group's `max_members` must be an integer between 2 and 50 inclusive.
- **Rule 2 (Unique Membership):** A user may have at most one record in `group_memberships` per study group (enforced via database unique constraint `uq_group_user`).
- **Rule 3 (Temporal Sanity):** Study sessions must satisfy $\text{end_time} > \text{start_time} > \text{current_timestamp}$.
- **Rule 4 (Owner Succession):** A group creator is permanently assigned role `'owner'`; deletion of a study group cascades to delete associated sessions, memberships, and resources.

---

6. Requirements Traceability Matrix Summary

Requirement ID	Module / Feature Name	Priority	Associated Use Case	Source Code Component	Primary Test Case ID	Test Status
FR-001	Student Registration	Must Have	`UC-01`	`auth.controller.js`, `auth.service.js`	`TC-001`, `TC-002`	**PASS**
FR-002	JWT Authentication	Must Have	`UC-01`	`auth.middleware.js`, `jwt.util.js`	`TC-003`, `TC-004`	**PASS**
FR-003	Password Hashing (bcrypt)	Must Have	`UC-01`	`auth.service.js`	`TC-017`	**PASS**
FR-004	Profile Metadata Setup	Must Have	`UC-02`	`user.controller.js`, `user.repository.js`	`TC-005`	**PASS**
FR-005	Weekly Availability JSONB	Must Have	`UC-02`	`user.repository.js`, `user_profiles`	`TC-005`	**PASS**
FR-006	Study Group Creation	Must Have	`UC-04`	`group.controller.js`, `group.service.js`	`TC-006`, `TC-007`	**PASS**
FR-007	Catalog Search & Filter	Must Have	`UC-03`	`group.controller.js`, `group.repository.js`	`TC-008`	**PASS**
FR-008	Join Request Submission	Must Have	`UC-04`	`group.controller.js`, `group.repository.js`	`TC-009`	**PASS**
FR-009	Owner Review & Approval	Must Have	`UC-08`	`group.service.js`, `OwnerReviewPanel.tsx`	`TC-010`	**PASS**
FR-010	Group Capacity Enforcement	Must Have	`UC-08`	`group.repository.js` (Row Locking)	`TC-010`	**PASS**
FR-011	Algorithmic Match Scoring	Should Have	`UC-05`	`matching.service.js`	`TC-011`, `TC-012`	**PASS**
FR-012	Ranked Recommendations	Should Have	`UC-05`	`matching.controller.js`, `RecommendationsPage.tsx`	`TC-012`	**PASS**
FR-013	Study Session Creation	Must Have	`UC-09`	`session.controller.js`, `session.service.js`	`TC-013`	PLANNED (Sprint 3)
FR-014	Session Time Validation	Must Have	`UC-09`	`session.validation.js`	`TC-014`	PLANNED (Sprint 3)
FR-015	Collision Detection Guard	Should Have	`UC-09`	`scheduling.service.js`	`TC-014`	PLANNED (Sprint 3)
FR-016	Interactive Calendar View	Should Have	`UC-10`	`CalendarPage.tsx`, `calendar.service.js`	`TC-013`	PLANNED (Sprint 3)
FR-017	Member RSVP Tracking	Should Have	`UC-06`	`session.controller.js`	`TC-015`	PLANNED (Sprint 3)
FR-018	Attendance Check-In	Could Have	`UC-06`	`session.controller.js`	`TC-015`	PLANNED (Sprint 3)
FR-019	Shared Academic Resources	Should Have	`UC-10`	`resource.controller.js`, `resource.service.js`	`TC-016`	PLANNED (Sprint 3)
FR-020	Resource Access Control	Should Have	`UC-10`	`resource.middleware.js`	`TC-016`	PLANNED (Sprint 3)
NFR-001	Sub-200ms API Latency	High	System	PostgreSQL GIN Indexes, Connection	`TC-018`	**PASS**

Requirement ID	Module / Feature Name	Priority	Associated Use Case	Source Code Component	Primary Test Case ID	Test Status
				Pool		
NFR-003	Responsive UI Layout	High	System	Tailwind CSS, React Viewports	`TC-019`	**PASS**
NFR-004	3NF Relational Integrity	High	System	Foreign Key Cascades, `uq_group_user`	`TC-020`	**PASS**